Shortest XOR Straight-Line Programs for 3×3 Matrix Multiplication: Benchmark Description

Greg Sidebottom
logic repository, https://github.com/gsidebottom/logic

Abstract—We contribute SAT instances deciding "do k XOR additions suffice?" for the output sides of rank-23 bilinear 3×3 matrix-multiplication schemes. Each boundary instance is a live mathematical question: an UNSAT answer at the right k lower-bounds the integer additive complexity of a scheme class (an optimality certificate). The exact minima are now independently known — via the transposition principle, which just produced the first sub-56 scheme (55 additions) — so these instances carry known ground-truth answers yet remain hard for every solver we tried. The family's AND-guarded parity structure defeats current XOR/Gaussian reasoning while remaining genuinely hard for plain CDCL: no tested configuration (kissat, CaDiCaL, CryptoMiniSat plain and native-XOR, Z3 word-level, portfolios with symmetry breaking) decides the calibrated boundary cells within 10–30 minutes.

I. Problem origin

A bilinear scheme multiplying two 3×3 matrices with 23 products [8] evaluates 23 products of linear forms and combines them into the 9 output entries. Its additive complexity — binary $\pm$ additions in a straight-line program, negation free, no change of basis — has fallen rapidly: 60 [4], 59 [3], 58 [2], and the current record 56 [1]. Sun's $56 = 13 + 13 + 30$ splits into two provably optimal input sides and an output side of 30 additions computing the 9 output forms from the 23 products.

55 additions are now achieved (13+14+28 on class i19w225c4efh), by minimizing the output side exactly via the transposition principle — the record was 56 (Sun 2026). That also yields, for every scheme, the exact output-side minimum in milliseconds, so these instances now carry known ground-truth answers (the SAT/UNSAT boundary is independently pinned) while remaining hard for every SAT/SMT solver tried; the open target is now 54.

Any $\mathbb{Z}$-coefficient output-side program reduces mod 2 to an XOR straight-line program, so the decision problem

> SLP(k): do k XOR additions suffice to compute the 9 given output forms (vectors in $\mathrm{GF}(2)^{23}$) from the 23 inputs?

yields sound lower bounds: UNSAT at k proves the integer output side needs $\geq k{+}1$ additions, and UNSAT at $k = 27$ on the right cells (§III) closes entire equivalence classes for 55. These instances continue the tradition of matrix-multiplication benchmarks in past competitions [5], with a twist of independent solver interest: the parity constraints are AND-guarded, which prevents CryptoMiniSat's Gaussian elimination from engaging, while plain CDCL faces genuine parity hardness.

II. Encoding

SLP synthesis in the style of Fuhs–Schneider-Kamp [6], simplified by unit-vector inputs. For steps $t = 1, \ldots, k$:

- Source selection. Step t selects exactly two sources among the 23 inputs and steps $1..t{-}1$ (sequential counter for the exactly-two constraint).
- Values. Value bits are parity-defined:

$$x_{t,i} \;=\; sel_{t,\mathrm{base}_i} \;\oplus\; \bigoplus_{j<t} \big(sel_{t,\mathrm{step}_j} \wedge x_{j,i}\big).$$

  Unit-vector inputs make the base contribution a single literal. The AND-guarded parities are materialized as Tseitin chains (plain CNF) or native x-lines (CryptoMiniSat extension); the generator emits both.
- Outputs. Each of the 9 forms must equal some step value (selector-guarded bit equalities; weight-1 forms may match an input).
- Symmetry breaking (optional, default on): all step values nonzero; adjacent independent steps (step $t{+}1$ does not consume step t) must have strictly lexicographically increasing values. Sound: swapping adjacent independent steps preserves validity, so every program normalizes by bubble sort, and padding above the minimum survives as a dependent chain. Dead-step elimination is deliberately not used: with it, satisfiability is not monotone in k (odd-length padding can force a dead step), which corrupts minimum-finding by descent.

At $k = 29$ with symmetry breaking: 25,799 variables, 91,426 clauses.

III. Instances and empirical hardness

An instance is determined by (output tensor of a scheme representative, k). Tensors come from the public database and the record chain; the de Groote action makes the output form-set depend only on the pair (R, P) of $\mathrm{GL}(3, 2)$ matrices; each of the $168 \times 168 = 28{,}224$ pairs is a cell (one concrete output-side instance), so every class supplies $\sim$28k cells — a practically unlimited supply, with k the hardness dial:

- SAT phase ($k \geq$ minimum): minutes at minimum + 1; witnesses are extracted and replay-verified by the generator.
- Deep UNSAT (k well below minimum): easy.
- Boundary ($k \in \{\min -1, \min\}$): hard. On the calibrated seed cells no tested configuration decides the boundary within 10–30 minutes (Apple M4 Pro, one core per solver): kissat (600 s and 1800 s), CaDiCaL (600 s), CryptoMiniSat 5 (615 s, plain and native-XOR), Z3 4.16 on a word-level QF_BV formulation (600 s), and a kissat+CaDiCaL+CMS portfolio with symmetry breaking (900 s).

TABLE I
Calibrated seed cells (SAT witnesses verified).

cell	weight	GF(2) min
sun56 output side (record scheme)	49	$\in \{29, 30\}$
cn120 output side (C=28 rep)	60	$\in \{27, 28\}$

Deciding a boundary certifies a per-class optimum: e.g. an UNSAT certificate at the transposition-computed minimum proves that a class's output side cannot be smaller, and (with the input-side exhaustions) that its total is optimal — a DRAT-checkable optimality theorem for a fixed class, now that a 55 scheme is known and the target is 54.

IV. Proposed benchmark set

Twenty instances spanning the phases: the two seed cells at $k \in \{\min -1, \min, \min +1\}$; the identity cells of the two other known 56-addition classes (i12w219c23ci, i19w225c4efh) at their boundaries; and four fat-sides window cells of the record class at $k = 27$; each with and without symmetry breaking. All are emitted by the C-side lower-bound generator (cxlb.py):

```
python3 matmul/cxlb.py --bits SCHEME.bits \
    --k K --dump out.cnf
python3 matmul/cxlb.py --bits SCHEME.bits \
    --k K --dump out.xnf   # native XOR
```

(–no-sb disables symmetry breaking; all scheme bits files are committed in the repository.)

V. Availability

Generator, scheme inputs, verification tooling, and research notes: https://github.com/gsidebottom/logic. Submitted instances may be used freely under the competition's standard terms.

Acknowledgments

Built in an extended interactive collaboration with Claude (Anthropic; the Fable 5 and Opus 4.8 models), which implemented the generators and verifiers under the author's direction and review; all instances and certificates are mechanically checkable.

References

[1] Y. Sun. An Exact 56-Addition, Rank-23 Scheme for General 3×3 Matrix Multiplication. arXiv:2604.27645, 2026.

[2] A. I. Perminov. A 58-Addition, Rank-23 Scheme for General 3×3 Matrix Multiplication. arXiv:2512.21980, 2025.

[3] E. Mårtensson, P. Stankovski Wagner, J. Stapleton. A Rank 23 Algorithm for Multiplying 3×3 Matrices with an Arithmetic Complexity of 59. arXiv:2601.05272, 2025.

[4] J. Stapleton. A 60-Addition, Rank-23 Scheme for Exact 3×3 Matrix Multiplication. arXiv:2508.03857, 2025.

[5] M. J. H. Heule, M. Kauers, J. Seidl. Local search for fast matrix multiplication. SAT 2019; J. Symbolic Computation 104, 2021.

[6] C. Fuhs, P. Schneider-Kamp. Synthesizing shortest linear straight-line programs over GF(2). SAT 2010.

[7] J. Boyar, R. Peralta. A new combinational logic minimization technique with applications to cryptology. SEA 2010.

[8] J. Laderman. A noncommutative algorithm for multiplying 3×3 matrices using 23 multiplications. Bull. AMS 82(1), 1976.